

Vibe Coding – Lab 4 Chat History

Student: Yashwanth Gowda T G

SRN: PES1UG25AM818

Project: 63_anvil_dodge

Assigned Repository: SETAPESU26/63_anvil_dodge

Personal Submission Repository: github.com/yashwanth00000/LAB1_AM818_SE

Task 1 – Inspect README and Understand the Existing Project

Vibe Coding Prompt

Before changing anything, inspect the complete Anvil Dodge repository and README.

Do not modify code yet.

Identify:

- Project entry point.
- Main game loop.
- Player class and movement logic.
- Anvil/hazard class and spawning logic.
- Collision detection.
- Game Over state.
- Existing score/timer/state variables.
- Rendering/UI code.
- All features explicitly required by the README.

Then give me:

1. A short architecture summary.
2. The files that need modification for each README task.
3. Potential risks or dependencies between tasks.

Do not implement any feature in this step.

Response / Result

The project structure and existing game flow were inspected before implementation. The main game loop, player movement, anvil behavior, collision/state handling, and rendering responsibilities were identified. The implementation plan separates each requested feature so that each change can be committed independently.

Task 2 – Run Baseline and Identify Existing Bugs

Vibe Coding Prompt

Run the existing Anvil Dodge game before making changes.

Focus only on finding existing problems.

Test:

- Player movement.
- Left/right screen boundaries.
- Anvil spawning.
- Falling speed.
- Player-anvil collision.
- Game Over behavior.
- Score behavior.
- Restart behavior.

Do not add features yet.

Report:

- What works.

- What is broken.
- Which behavior should be preserved after modifications.

Response / Result

The baseline behavior was used as the reference point before modifications. Existing gameplay behavior was kept as the baseline so later changes could be tested without unintentionally changing unrelated mechanics.

Task 3 – Fix Player Off-Screen Boundary Bug

Vibe Coding Prompt

Inspect the existing Anvil Dodge Python/Pygame project and focus only on the player boundary bug.

The player can move beyond the visible left or right edge of the game window.

Requirements:

- Keep existing controls unchanged.
- Prevent the player from leaving the visible game area.
- Keep the complete player sprite visible.
- Preserve collision detection and existing gameplay.
- Do not implement other README tasks.
- Make the smallest clean change necessary.
- Explain the cause, modified file/function, and fix.

Response / Result

The horizontal player position was constrained to the visible game boundaries while preserving the existing controls and gameplay flow.

Commit: Fix player off-screen boundary bug

Task 4 – Implement Dynamic Difficulty Scaling

Vibe Coding Prompt

Implement only dynamic difficulty scaling.

Inspect how anvils are spawned and how their falling velocity is controlled.

Requirements:

- Begin at the current normal difficulty.
- Gradually increase difficulty as survival time increases.
- Increase anvil speed and/or spawning frequency progressively.
- Avoid sudden unfair difficulty jumps.
- Preserve the player boundary fix.
- Preserve collision and Game Over behavior.
- Do not implement other features yet.
- Explain the scaling logic and adjustable values.

Response / Result

Difficulty progression was added around the existing anvil system so the game becomes harder as the player survives longer, while keeping the earlier boundary and collision behavior intact.

Commit: Implement dynamic difficulty scaling

Task 5 – Add Speed-Based Anvil Warning Tints

Vibe Coding Prompt

Implement only speed-based visual warning tints for anvils.

Requirements:

- Slow anvils keep their normal appearance.
- Medium-speed anvils receive a warning tint.
- High-speed anvils receive a stronger danger tint.
- Base the tint on current falling velocity.

- Use clear velocity thresholds.
- Do not change collision behavior.
- Do not change player movement.
- Preserve all previous tasks.
- Explain the thresholds and rendering logic.

Response / Result

Anvil rendering was connected to falling velocity so faster anvils become visually more dangerous without changing collision or movement behavior.

Commit: Add speed-based anvil warning tints

Task 6 – Add Survival Timer

Vibe Coding Prompt

Add only a visible survival timer.

Requirements:

- Start when a new game begins.
- Update continuously during active gameplay.
- Stop when Game Over occurs.
- Display elapsed time clearly on the HUD.
- Reset when the player starts a new run.
- Do not create a second game loop.
- Preserve all existing gameplay.
- Explain where the timer state is stored and rendered.

Response / Result

A survival-time HUD was added and tied to the active game state. It stops at Game Over and is reset for a fresh run without introducing a second game loop.

Commit: Add survival timer

Task 7 – Add Score Updates

Vibe Coding Prompt

Add only the score feature.

Requirements:

- Display the current score clearly.
- Increase the score while the player successfully survives/dodges anvils.
- Stop score updates on Game Over.
- Reset score to zero on a new run.
- Preserve the timer and all previous features.
- Do not alter player movement or collision behavior.
- Use a simple and understandable scoring rule.
- Explain exactly when the score changes.

Response / Result

A visible score HUD was added with a simple survival-based scoring rule. Score updates stop at Game Over and reset when a fresh run begins.

Commit: Add score system

Task 8 – Improve Game Over Screen

Vibe Coding Prompt

Improve only the Game Over presentation.

Requirements:

- Clearly display GAME OVER.
- Show the final score.
- Show the final survival time.
- Stop normal gameplay.
- Stop timer and score updates.
- Keep the window open until the player chooses a restart/exit action.
- Do not remove existing collision or difficulty behavior.
- Keep the implementation inside the existing game architecture.

Response / Result

The Game Over state now presents the final run information while preventing further gameplay updates. The final score and survival time remain visible until the user chooses the next action.

Commit: Improve Game Over screen

Task 9 – Implement Complete Game Reset

Vibe Coding Prompt

Implement a complete and reliable restart system.

When the player restarts after Game Over, reset all run-specific state:

- Player position and velocity.
- Active anvils.
- Score.
- Survival timer.
- Difficulty progression.
- Particles.
- Screen shake.
- Game Over state.
- Any temporary effects or timers.

Requirements:

- The restarted game must behave exactly like a fresh game.
- Repeated restart cycles must not duplicate objects, timers or effects.
- Preserve all existing gameplay features.
- Do not rewrite unrelated code.
- Explain the reset sequence and the state variables that are reinitialized.

Response / Result

The restart flow was designed as a complete run-state reset. Player state, hazards, score, timer, difficulty and temporary effects are cleared before the new run starts, preventing state from leaking between games.

Commit: Add complete game reset

Task 10 – Test Repeated Game Over → Restart Cycles

Vibe Coding Prompt

Perform a focused regression test of the complete game.

Test this sequence at least three times:

START

- play normally
- trigger Game Over
- verify final score/time
- restart
- verify clean initial state
- play again
- trigger Game Over again

Check specifically for:

- Duplicate anvils.
- Duplicate timers.
- Score continuing from the previous run.
- Difficulty starting too high.
- Particles or screen shake remaining active.
- Player starting at the wrong position.
- Timer continuing after Game Over.

Do not add new features during this test. Report any bugs found and fix only those bugs.

Response / Result

The restart cycle was treated as a regression test. The important checks are that all run-specific state returns to its initial values and that repeated restarts do not accumulate timers, hazards, particles or other temporary state.

Task 11 – Review Code Quality Without Changing Behavior

Vibe Coding Prompt

Review the completed Anvil Dodge code for unnecessary duplication, dead code, confusing variable names and unsafe state handling.

Requirements:

- Do not change gameplay behavior.
- Do not add new features.
- Keep the current architecture.
- Remove only clearly unnecessary code.
- Improve readability where it is safe.
- Do not introduce external dependencies.
- Explain every cleanup change before applying it.
- Verify that all existing features remain functional after cleanup.

Response / Result

The final review should focus on readability and safe state handling rather than introducing additional functionality. Existing behavior must remain unchanged after cleanup.

Task 12 – Final Validation and Submission Check

Vibe Coding Prompt

Perform a final validation of the completed Anvil Dodge project.

Verify:

1. Player cannot leave the screen.
2. Difficulty increases progressively.
3. Fast anvils show warning tints.
4. Timer starts, updates and stops correctly.
5. Score updates and stops correctly.
6. Game Over displays final score and time.

7. Restart completely resets the run.
8. Repeated restarts do not duplicate state.
9. Existing controls still work.
10. The game can be launched normally.

Also check that the submission contains:

- Before video.
- After video.
- Completed code.
- Vibe Coding chat history PDF.

Do not add new functionality during this validation step.

Response / Result

The final validation checklist covers the implemented gameplay features and the required Lab-4 submission artifacts. Any fixes found during validation should be committed separately from the feature commits.

Final Commit Sequence

```
git add .
git commit -m "Fix player off-screen boundary bug"
```

```
git add .
git commit -m "Implement dynamic difficulty scaling"
```

```
git add .
git commit -m "Add speed-based anvil warning tints"
```

```
git add .
git commit -m "Add survival timer"
```

```
git add .
git commit -m "Add score system"
```

```
git add .
git commit -m "Improve Game Over screen"
```

```
git add .
git commit -m "Add complete game reset"
```

```
git push origin main
```

Submission Details

Student: Yashwanth Gowda T G

SRN: PES1UG25AM818

Personal Repository: github.com/yashwanth00000/LAB1_AM818_SE

Submission Folder: Lab-4/

Required: Before/before.mp4, After/after.mp4, Code/63_anvil_dodge, Chat_History/VibeCoding_Chat_History.pdf